

Privacy Preserving Data Mining

Yehuda Lindell

Department of Computer Science
Weizmann Institute of Science
Rehovot, ISRAEL.

`lindell@wisdom.weizmann.ac.il`

Benny Pinkas*

STAR Lab, Intertrust Technologies
4750 Patrick Henry Drive
Santa Clara CA 95054.

`bpinkas@intertrust.com`, `benny@pinkas.net`

Abstract

In this paper we introduce the concept of privacy preserving data mining. In our model, two parties owning confidential databases wish to run a data mining algorithm on the union of their databases, without revealing any unnecessary information. This problem has many practical and important applications, such as in medical research with confidential patient records.

Data mining algorithms are usually complex, especially as the size of the input is measured in megabytes, if not gigabytes. A generic secure multi-party computation solution, based on evaluation of a circuit computing the algorithm on the entire input, is therefore of no practical use. We focus on the problem of decision tree learning and use ID3, a popular and widely used algorithm for this problem. We present a solution that is considerably more efficient than generic solutions. It demands very few rounds of communication and reasonable bandwidth. In our solution, each party performs by itself a computation of the same order as computing the ID3 algorithm for its own database. The results are then combined using efficient cryptographic protocols, whose overhead is only logarithmic in the number of transactions in the databases. We feel that our result is a substantial contribution, demonstrating that secure multi-party computation can be made practical, even for complex problems and large inputs.

Key words: Secure two-party computation, Oblivious transfer, Oblivious polynomial evaluation, Data mining, Decision tree.

*Most of this work was done while at the Weizmann Institute of Science and the Hebrew University of Jerusalem, and was supported by an Eshkol grant of the Israel Ministry of Science.

1 Introduction

We consider a scenario where two parties having private databases wish to cooperate by computing a data mining algorithm on the union of their databases. Since the databases are confidential, neither party is willing to divulge any of the contents to the other. We show how the involved data mining problem of decision tree learning can be efficiently computed, with no party learning anything other than the output itself. We demonstrate this on ID3, an algorithm widely used and implemented in many real applications.

Data Mining. Data mining is a recently emerging field, connecting the three worlds of Databases, Artificial Intelligence and Statistics. The information age has enabled many organizations to gather large volumes of data. However, the usefulness of this data is negligible if “meaningful information” or “knowledge” cannot be extracted from it. Data mining, otherwise called *knowledge discovery*, attempts to answer this need. In contrast to standard statistical methods, data mining techniques search for *interesting* information without demanding a priori hypotheses. As a field, it has introduced new concepts and algorithms such as association rule learning. It has also applied known machine-learning algorithms such as inductive-rule learning (e.g., by decision trees) to the setting where very large databases are involved. Data mining techniques are used in business and research and are becoming more and more popular with time.

Confidentiality Issues in Data Mining. A key problem that arises in any en masse collection of data is that of *confidentiality*. The need for secrecy is sometimes due to law (e.g. for medical databases) or can be motivated by business interests. However, sometimes there can be mutual gain by *sharing* of data. A key utility of large databases today is research, whether it be scientific, or economic and market oriented. The medical field has much to gain by pooling data for research; as can even competing businesses with mutual interests. Despite the potential gain, this is not possible due to confidentiality issues which arise.

We address this question and show that practical solutions are possible. Our scenario is the following:

There are two parties P_1 and P_2 that own databases D_1 and D_2 . The parties wish to apply a data-mining algorithm to the joint database $D_1 \cup D_2$ without revealing any unnecessary information about their individual databases. That is, the only information learned by P_1 about D_2 is that which can be learned from the output, and vice versa. We do not assume any “trusted” third party who computes the joint output.

Very Large Databases and Efficient Computation. We have described a model which is exactly that of multi-party computation. Therefore, there exists a secure solution for *any* functionality (Goldreich et. al. in [13]). As we discuss in Section 1.1, due to the fact that these solutions are generic, they are highly inefficient. In our case where the inputs are very large and the algorithms reasonably complex, they are far from practical.

It is clear that any reasonable solution must have the individual parties do the majority of the computation independently. Our solution is based on this guiding principle and in fact, the number of bits communicated is dependent on the number of transactions by a logarithmic factor only.

Semi-Honest Parties. In any multi-party computation setting, a *malicious* party can always alter his input. In the data-mining setting, this fact can be very damaging as an adversarial party may define his input to be the empty database. Then, the output obtained is the result of the algorithm on the other party’s database alone. Although this attack cannot be prevented, we would like to limit attacks by malicious parties to altering their input only. However, for this initial work we assume that the parties are *semi-honest* (also termed *passive*). That is, they follow the protocol as it is defined, but may record all intermediate messages sent in an attempt to later derive additional

information. We leave the question of an efficient solution to the malicious party setting for future work. In any case, as was described above, malicious parties cannot be prevented from obtaining meaningful confidential information and therefore a certain level of trust is anyway needed between the parties. We remark that the semi-honest model is often a realistic one; that is, deviating from a specified program which may be buried in a complex application is a non-trivial task.

1.1 Related Work

Secure two party computation was first investigated by Yao [21], and was later generalized to multi-party computation in [13, 2, 5]. These works all use a similar methodology: the function F to be computed is first represented as a combinatorial circuit, and then the parties run a short protocol for every gate in the circuit. While this approach is appealing in its generality and simplicity, the protocols it generates depend on the size of the circuit. This size depends on the size of the input (which might be huge as in a data mining application), and on the complexity of expressing F as a circuit (for example, a multiplication circuit is quadratic in the size of its inputs). We stress that secure computation of small circuits with small inputs can be *practical* using the [21] protocol.¹

One of the first examples of a secure two party protocol for a specific problem is that of oblivious transfer [20, 8], which proved to be a very useful primitive for many applications as for circuit evaluation [14]. A more recent example of the efficient secure computation of a much more complex function is the distributed generation of RSA keys [3, 11]. Another example is Private Information Retrieval (PIR) protocols [6], which enable one party to query a database held by another party, while keeping the queries private. The emphasis of PIR protocols is on the communication overhead, which must be smaller than the size of the database. The computational overhead of PIR protocols is typically very high (although polynomial).

There is a major difference between the protocol described in this paper and these above examples. While previous protocols were efficient (polynomial) in the size of their inputs, this property does not suffice for data mining applications, as the input consists of huge databases. In the protocol presented here, most of the computation is done individually by each of the parties. They then engage in a few secure circuit evaluations on very small circuits. We obtain very few rounds of communication with bandwidth which is practical for even very large databases.

1.2 Outline

The next section describes the problem of classification and a widely used solution to it, the ID3 algorithm for decision tree learning. Following this, Section 3 presents the security definition and Section 4 describes the cryptographic tools used in the solution. Section 5 contains the protocol itself and in Section 6 the main subprotocol that privately computes random shares of $f(v_1, v_2) \stackrel{\text{def}}{=} (v_1 + v_2) \ln(v_1 + v_2)$ is described. Finally, in Section 7 we demonstrate the efficiency of our protocol and compare the complexity to that of generic solutions.

2 Classification by Decision Tree Learning

This section briefly describes the machine learning and data mining problem of *classification* and ID3, a well-known algorithm for it. The presentation here is rather simplistic and very brief and we refer the reader to Mitchell [15] for an in-depth treatment of the subject. The ID3 algorithm for generating decision trees was first introduced by Quinlan in [19] and has since become a very popular learning tool.

¹The [21] protocol requires only two rounds of communication. Furthermore, since the circuit and inputs are small, the bandwidth is not too great and only a reasonable number of oblivious transfers need be executed.

2.1 The Classification Problem

The aim of a classification problem is to classify transactions into one of a discrete set of possible categories. The input is a structured database comprised of attribute-value pairs. Each row of the database is a *transaction* and each column is an *attribute* taking on different values. One of the attributes in the database is designated as the *class* attribute; the set of possible values for this attribute being the classes. We wish to predict the class of a transaction by viewing only the non-class attributes. This can thus be used to predict the class of new transactions for which the class is unknown.

For example, a bank may wish to conduct credit risk analysis in an attempt to identify non-profitable customers before giving a loan. The bank then defines “Profitable-customer” (obtaining values “yes” or “no”) to be the class attribute. Other database attributes may include: Home-Owner, Income, Years-of-Credit, Other-Delinquent-Accounts and other relevant information. The bank is interested in learning rules such as:

If (Other-Delinquent-Accounts = 0) and (Income > 30k or Years-of-Credit > 3)
 then Profitable-customer = YES [accept credit-card application]

A collection of such rules covering all possible transactions can then be used to classify a *new* customer as potentially profitable or not. The classification may also be accompanied with a probability of error. Not all classification techniques output a set of meaningful rules, we have brought just one example here.

Another example is to attempt to predict whether a woman is at high risk for an Emergency Caesarean Section, based on data gathered during the pregnancy. There are many useful examples and it is not hard to see why this type of learning or mining task has become so popular.

The success of an algorithm on a given data set is measured by the percentage of new transactions correctly classified. Although this is an important data mining (and machine learning) issue, we do not go into it here.

2.2 Decision Trees and the ID3 Algorithm

A decision tree is a rooted tree containing nodes and edges. Each internal node is a test node and corresponds to an attribute; the edges leaving a node correspond to the possible values taken on by that attribute. For example, the attribute “Home-Owner” would have two edges leaving it, one for “Yes” and one for “No”. Finally, the leaves of the tree contain the *expected* class value for transactions matching the path from the root to that leaf.

Given a decision tree, one can predict the class of a new transaction t as follows. Let the attribute of a given node v (initially the root) be A , where A obtains possible values $a_1, \dots, a_m$. Then, as described, the m edges leaving v are labeled $a_1, \dots, a_m$ respectively. If the value of A in t equals a_i , we simply go to the son pointed to by a_i . We then continue recursively until we reach a leaf. The class found in the leaf is then assigned to the transaction.

We use the following notation:

- R : the set of *attributes*
- C : the *class* attribute
- T : the set of *transactions*

The ID3 algorithm assumes that each attribute is categorical, that is containing discrete data only, in contrast to continuous data such as age, height etc.

The principle of the ID3 algorithm is as follows:

The tree is constructed top-down in a recursive fashion. At the root, each attribute is tested to determine how well it alone classifies the transactions. The “best” attribute (to be discussed below) is then chosen and the remaining transactions are partitioned by it. ID3 is then recursively called on each partition (which is a smaller database containing only the appropriate transactions and without the splitting attribute). See Figure 1 for a description of the ID3 algorithm.

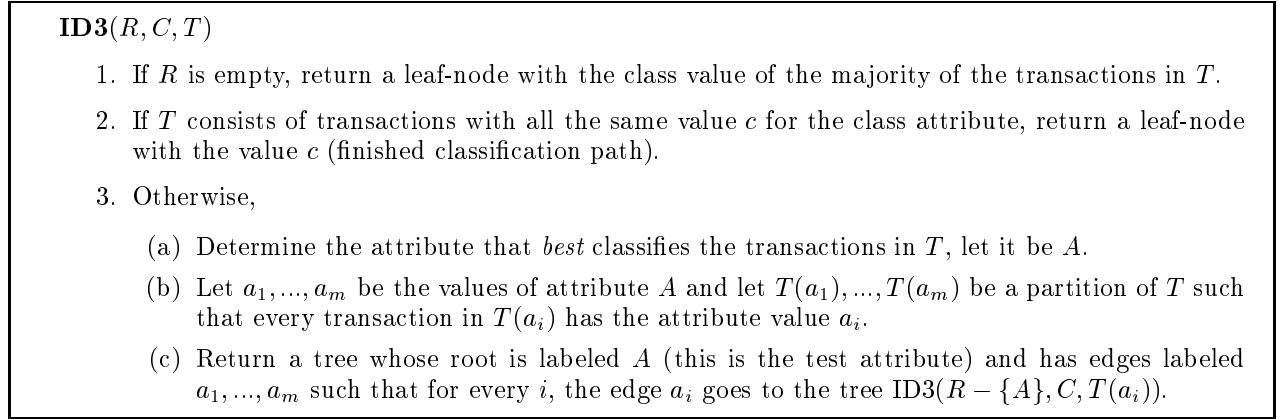


Figure 1: The ID3 Algorithm for Decision Tree Learning

What remains is to explain how the *best* predicting attribute is chosen. This is the central principle of ID3 and is based on information theory. The entropy of the class attribute clearly expresses the difficulty of prediction. We know the class of a set of transactions when the class entropy for them equals zero. The idea is therefore to check which attribute reduces the information of the class-attribute by the most. This results in a greedy algorithm which searches for a small decision tree consistent with the database. The bias favoring short descriptions of a hypothesis is based on Occam’s razor. As a result of this, decision trees are usually relatively small, even for large databases.

The exact test for determining the best attribute is defined as follows. Let $c_1, \dots, c_\ell$ be the class-attribute values. Let $T(c_i)$ be the set of transactions with class c_i . Then the information needed to identify the class of a transaction in T is the entropy, given by:

$$H_C(T) = \sum_{i=1}^{\ell} -\frac{|T(c_i)|}{|T|} \log \frac{|T(c_i)|}{|T|}$$

Let A be a non-class attribute. We wish to quantify the information needed to identify the class of a transaction in T *given* that the value of A has been obtained. Let A obtain values $a_1, \dots, a_m$ and let $T(a_j)$ be the transactions obtaining value a_j for A . Then, the conditional information of T given A is given by:

$$H_C(T|A) = \sum_{j=1}^m \frac{|T(a_j)|}{|T|} H_C(T(a_j))$$

Now, for each attribute A the information-gain is defined by

$$\text{Gain}(A) \stackrel{\text{def}}{=} H_C(T) - H_C(T|A)$$

The attribute A which has the maximum gain over all attributes in R is then chosen.²

²Note that the gain measure biases attributes with many values and another measure called the *Gain Ratio* is therefore sometimes used. We present the simpler version here.

Extensions of ID3. Since its inception there have been many extensions to the original algorithm, the most well-known being C4.5. We now briefly describe some of these extensions. One of the immediate shortcomings of ID3 is that it only works on discrete data, whereas most databases contain continuous data. A number of methods enable the incorporation of continuous-value attributes, even as the class attribute. Other extensions include handling missing attribute values, alternative measures for selecting attributes and reducing the problems of overfitting with pruning.

We consider only the simpler ID3 algorithm and leave extensions to more advanced versions such as C4.5 for future work.

See Appendix A for a short example of a database and the resulting decision tree.

2.3 The $ID3_\delta$ Approximation

The ID3 algorithm chooses the “best” predicting attribute by comparing entropies that are given as real numbers. If at a given point, two entropies are very close together, then the two (different) trees resulting from choosing one attribute or the other are expected to have almost the same predicting capability. Formally stated, let δ be some small value. Then, for a pair of attributes A_1 and A_2 , we say that A_1 and A_2 have δ -equivalent information gains if

$$|H_C(T|A_1) - H_C(T|A_2)| < \delta$$

This definition gives rise to an approximation of ID3. Denote by $ID3_\delta$ the set of all possible trees which are generated by running the ID3 algorithm, and choosing either A_1 or A_2 in the case that they have δ -equivalent information gains. We actually present a protocol for secure computation of a specific algorithm $ID3_\delta \in ID3_\delta$, in which the choice of A_1 or A_2 is implicit by an approximation that is used instead of the log function. The value of δ influences the efficiency, but only by a logarithmic factor.

Note that any naive implementation of ID3 that computes the logarithm function to a predefined precision level has an approximation error, and therefore essentially computes $ID3_\delta$. However, a more elaborate implementation of ID3 can resolve this problem as follows. First, the information gain for each attribute is computed using a predefined precision level that results in a log approximation of error at most δ . Then, if two information gains compared are found to be within δ' of each other (where δ' is the precision error of the information gain resulting from a δ error in the log function), then the information gains are recomputed using a higher precision level for the log function. This is continued until we are ensured that the attribute with the maximal information gain is found. We do not know how to achieve similar accuracy in a privacy preserving implementation.

3 Security Definition – Private Computation of Functions

The model for this work is that of general multi-party computation, more specifically between two *semi-honest* parties. Our formal definitions here are according to Goldreich in [12]. We now present in brief the definition for general two-party computation of a functionality with *semi-honest* parties only.

Formalizations of Privacy. There are two equivalent approaches for formalizing privacy in computation with semi-honest parties. Both capture the notion that whatever can be computed by a party participating in the protocol can be computed based on the party’s input and output only. Since by the problem definition the parties have access to their input and output, this means that no additional information is learned.

The first approach is according to the simulation paradigm and says that a party’s *view* of the computation may be simulated given only the input and output. The second approach compares

the computation to an *ideal* computation involving an incorruptible trusted third party to whom the parties send their inputs. The trusted party then computes the function and returns to each their respective output. The protocol is secure if any adversary computing a function based on the real-model computation can be converted into an adversary computing the same function in the ideal model. As we have mentioned the two approaches are equivalent in the semi-honest party setting. The first is however simpler and we therefore base our presentation on the simulation paradigm in this paper.

Formal Definition. The following definitions are taken from [12]. We begin with the following notation:

- Let $f : \{0,1\}^* \times \{0,1\}^* \mapsto \{0,1\}^* \times \{0,1\}^*$ be a functionality where $f_1(x, y)$ (*resp.*, $f_2(x, y)$) denotes the first (*resp.*, *second*) element of $f(x, y)$ and let Π be a two-party protocol for computing f .
- The view of the first (*resp.*, *second*) party during an execution of Π on (x, y) , denoted $\text{view}_1^\Pi(x, y)$ (*resp.*, $\text{view}_2^\Pi(x, y)$), is $(x, r^1, m_1^1, \dots, m_t^1)$ (*resp.*, $(y, r^2, m_1^2, \dots, m_t^2)$) where r^1 (*resp.*, r^2) represents the outcome of the first (*resp.*, *second*) party's *internal* coin tosses, and m_i^1 (*resp.*, m_i^2) represents the i 'th message it has received.
- The output of the first (*resp.*, *second*) party during an execution of Π on (x, y) is denoted $\text{output}_1^\Pi(x, y)$ (*resp.*, $\text{output}_2^\Pi(x, y)$), and is implicit in the party's view of the execution.

We note that in the case of ID3_δ itself, we have $f_1 = f_2 = \text{ID3}_\delta$. However, in the subprotocols that we use it is often the case that $f_1 \neq f_2$.

Definition 1 (privacy w.r.t. semi-honest behavior): *For a functionality f , we say that Π privately computes f if there exist probabilistic polynomial time algorithms, denoted S_1 and S_2 , such that*

$$\{(S_1(x, f_1(x, y)), f_2(x, y))\}_{x, y \in \{0,1\}^*} \stackrel{c}{=} \{(\text{view}_1^\Pi(x, y), \text{output}_2^\Pi(x, y))\}_{x, y \in \{0,1\}^*} \quad (1)$$

$$\{(f_1(x, y), S_2(y, f_2(x, y)))\}_{x, y \in \{0,1\}^*} \stackrel{c}{=} \{(\text{output}_1^\Pi(x, y), \text{view}_2^\Pi(x, y))\}_{x, y \in \{0,1\}^*} \quad (2)$$

where $\stackrel{c}{=}$ denotes computational indistinguishability.

Equations (1) and (2) state that the views of the parties can be simulated by a polynomial time algorithm given access to the party's *input and output only*. We emphasize that the parties here are semi-honest and the view is therefore exactly according to the protocol definition. We note that it is not enough for the simulator S_1 to generate a string indistinguishable from $\text{view}_1^\Pi(x, y)$. Rather, the *joint distribution* of the simulator's output and $f_2(x, y)$ must be indistinguishable from $(\text{view}_1^\Pi(x, y), \text{output}_2^\Pi(x, y))$. See [4, 12] for a discussion on why this is essential.

Private Data Mining. We now discuss issues specific to the case of multi-party computation where the inputs x and y are databases. Denote the two parties P_1 and P_2 and their respective private databases D_1 and D_2 . First, we assume that D_1 and D_2 have the same structure and that the attribute names are public.³ There is a somewhat delicate issue when it comes to the names of the possible values for each attribute. On the one hand, universal names must clearly be agreed upon in order to compute any joint function. On the other hand, even the *existence* of a certain attribute value in a database can be sensitive information. This problem can be solved

³This is essential for carrying out any joint computation in this setting.

by a pre-processing phase in which random value names are assigned to the values such that they are consistent in both databases. This is in itself a non-trivial problem but one that can be solved using similar techniques to those presented in this paper. We ignore this problem and assume that the attribute-value names are also public (as would be *after* the described random-mapping stage). Next, as we have discussed, each party should receive the output of some data mining algorithm on the union of their databases, $D_1 \cup D_2$. We note that in actuality we consider a *merging* of the two databases so that if the same transaction appears in both databases, then it appears twice in the merged database. Finally, assume that an upper-bound on the size of $|D_1 \cup D_2|$ is known and public.

Composition of Private Protocols. The protocol for privately computing $ID3_\delta$ is composed of many invocations of smaller private computations. In particular, we reduce the problem to that of privately computing smaller subproblems and show how to compose them together in order to obtain a complete $ID3_\delta$ solution. Although intuitively clear, this composition requires formal justification. We present a brief, informal discussion and refer the reader to Goldreich [12] for a complete, formal treatment.

Informally, consider *oracle-aided* protocols, where the queries are supplied by both parties. The *oracle-answer* may be different for each party, depending on its definition. Note that the oracles may be probabilistic. An oracle-aided protocol is said to *privately reduce* g to f if it privately computes g when using the oracle functionality f . The security of our solution relies heavily on the following intuitive theorem.

Theorem 2 (Composition Theorem for the semi-honest model, two parties): *Suppose that g is privately reducible to f and that there exists a protocol for privately computing f . Then, the protocol defined by replacing each oracle-call to f by a protocol that privately computes f , is a protocol for privately computing g .*

In the semi-honest case this theorem is easily obtained by simply plugging in the simulator for the private computation of the oracle functionality. Furthermore, it is easily generalized to the case where a number of oracle-functionalities $f_1, f_2, \dots$ are used in privately computing g . The composition theorem (for sequential composition) is also true for the malicious case, but is far more complex (see [4]).

3.1 Secure Computation of Approximations and of $ID3_\delta$

Our work takes $ID3_\delta$ as the starting point and security is guaranteed relative to the approximated algorithm, rather than to $ID3$ itself. We present a secure protocol for computing $ID3_\delta$. That is, P_1 can compute his view given D_1 and $ID3_\delta(D_1 \cup D_2)$ only (likewise P_2). However, this does *not* mean that $ID3_\delta(D_1 \cup D_2)$ reveals the “same” (or less) information as $ID3(D_1 \cup D_2)$ does. In fact, it is clear that although the computation of $ID3_\delta$ is secure, *different* information is revealed (intuitively though, no “more” information is revealed).⁴

The problem of secure distributed computation of approximations was introduced and discussed by Feigenbaum et. al. [10]. Their main motivation is a scenario in which the computation of an approximation to a function f might be considerably more efficient than the computation of f itself. The security definition requires that the approximation does not reveal more about the inputs than f does. In addition, the paper presents several general techniques for computing approximations, and efficient protocols for computing approximations of distances.

⁴Note that although our implementation approximates many invocations of the natural logarithm function, none of these approximations are revealed. The only approximation which becomes known to the parties is the final $ID3_\delta$ decision tree.

4 Cryptographic Tools

4.1 Oblivious Transfer

The notion of 1-out-2 oblivious transfer (OT_1^2) was suggested by Even, Goldreich and Lempel [8], as a generalization of Rabin’s “oblivious transfer” [20]. This protocol involves two parties, the *sender* and the *receiver*. The sender has two inputs $\langle X_0, X_1 \rangle$, and the receiver has an input $\sigma \in \{0, 1\}$. At the end of the protocol the receiver should learn X_σ and no other information, and the sender should learn nothing.

Very attractive non-interactive protocols for oblivious transfer were suggested by Bellare and Micali [1], which require only a single back and forth communication round between the parties. As a general rule, the computational overhead of an OT_1^2 protocol involves the computation of very few exponentiations. A recent result [17] describes how to reduce the amortized overhead of oblivious transfer, and is very important if many oblivious transfers must be computed in the same protocol. An additional result in [17] demonstrates how to compute non-interactive oblivious transfer of *strings* without using a function which is assumed to be a “random oracle”. Oblivious transfer protocols can be greatly simplified if the parties are assumed to be semi-honest, as they are in the application discussed in this paper. (For example, in the semi-honest case, the security of the [1] protocol does not rely on random oracles.)

4.2 Oblivious Evaluation of Polynomials

The oblivious polynomial evaluation problem involves a sender that has a polynomial P of degree k over some finite field $\mathcal{F}$ and a receiver with an element $z \in \mathcal{F}$ (the degree k of P is public). The receiver obtains $P(z)$ without learning anything else about the polynomial P and the sender learns nothing about z . This primitive was introduced in [16]. For our solution we use a new protocol [7] that requires $O(k)$ exponentiations in order to evaluate a polynomial of degree k (where the ‘ O ’ coefficient is very small). This is important as we work with low-degree polynomials. Following are the basic ideas of this protocol.

Let $P(x) = \sum_{i=0}^k a_i x^i$ and z be the sender and receiver’s respective inputs. The following protocol enables the receiver to compute $g^{P(z)}$, where g is a generator of a group in which the Decisional Diffie-Hellman assumption holds. The protocol is very simple since it is assumed that the parties are semi-honest. It can be converted to one which computes $P(z)$ using the methods of Paillier [18], who presented a trapdoor for computing discrete logs. Security against malicious parties can be obtained using proofs of knowledge. The protocol consists of the following steps:

- The receiver chooses a secret key s , and sends g^s to the sender.
- For $0 \leq i \leq k$, the receiver computes $c_i = (g^{r_i}, g^{s \cdot r_i} g^{z^i})$, where r_i is random. The receiver sends $c_0, \dots, c_k$ to the sender.
- The sender computes $C = \prod_{i=0}^k (c_i)^{a_i} = (g^R, g^{sR} g^{P(z)})$, where $R = \sum_{i=0}^k r_i a_i$. It then chooses a random value r and computes $C' = (g^R \cdot g^r, g^{sR} g^{P(z)} \cdot g^{sr})$ and sends it to the receiver.
- The receiver divides the second element of C' by the first element of C' raised to the power of s , and obtains $g^{P(z)}$.

Proof Sketch: We first show that the sender learns nothing of z from the protocol. The sender’s view consists of g^s for a random s and pairs $c_i = (g^{r_i}, g^{s \cdot r_i} g^{z^i})$ for random r_i ($0 \leq i \leq k$). The value of z clearly remains secret if $(g^{r_i}, g^{s \cdot r_i})$ is indistinguishable from (g^{r_i}, g^{α_i}) for random α_i , as z^i is masked by a pseudorandom value in each tuple. The Decisional Diffie-Hellman assumption states that for random a, b, c , the tuples (g^a, g^b, g^{ab}) and (g^a, g^b, g^c) are indistinguishable. In our

case we need that the two series of tuples $(g^{r_i}, g^s, g^{s \cdot r_i})$ and $(g^{r_i}, g^s, g^{\alpha_i})$ be indistinguishable. This is not exactly the same as the standard assumption as the second element g^s is repeated in many tuples. However, using a simple hybrid argument, it can be shown that this is indeed implied by the Decisional Diffie-Hellman assumption and thus the sender learns nothing of z .

We now show that the receiver learns nothing of the polynomial P . The receiver's view consists only of the message:

$$C' = (g^R \cdot g^r, g^{sR} \cdot g^{P(z)} \cdot g^{sr}) = (g^{R+r}, g^{sR+sr+P(z)})$$

Due to the “masking” by the random value r , the receiver can easily generate this message distribution solely from the output $g^{P(z)}$. This is done by choosing a random r' (representing the value $R + r$ which is uniformly distributed due to r 's randomness) and computing $s \cdot r' + P(z)$. The receiver's simulator can do this as s is derived from its own random string and $P(z)$ is given to the simulator. Now, it is clear that $(g^{r'}, g^{s \cdot r' + P(z)})$ is identically distributed to C' . We note that the simulation works also in the case that the receiver can compute the discrete log, and this completes the proof. ■

4.3 Oblivious Circuit Evaluation

The two party protocol of Yao [21] solves the following problem. There are two parties, a party A which has an input x , and a party B which has as input a function f and a combinatorial circuit that computes f . At the end of the protocol A outputs $f(x)$ and learns no other information about f , while B learns nothing at all. We employ this protocol for the case that f depends on two inputs x and y , belonging to A and B respectively. This is accomplished by having B simply hardwire his input y into the circuit (that is, B 's input is a function $f(\cdot, y)$ and A obtains $f(x, y)$ from the circuit).⁵

We describe Yao's protocol in detail in Appendix B. The overhead of the protocol involves (1) B sending to A tables of size linear in the size of the circuit, (2) A and B engaging in an oblivious transfer protocol for every input wire of the circuit, and (3) A computing a pseudo-random function a constant number of times for every gate. Therefore, the number of rounds of this protocol is constant (namely, two rounds using non-interactive oblivious transfer), and if the circuit is small (e.g., linear in the size of the input) then the main computational overhead is that of running the oblivious transfers.

Computing Random Shares. In our protocol, we are interested in using Yao's circuit evaluation for obtaining random shares r_1 and r_2 such that $r_1 + r_2 = f(x, y)$. This effect is easily achieved in the following way. B chooses a random r_2 and augments its input to include both y and r_2 . The circuit is defined so that it computes $r_1 = F(x, (y, r_2)) \stackrel{\text{def}}{=} f(x, y) - r_2$. Then, A and B have random shares summing to $f(x, y)$ as required.

5 The Protocol

The central idea of our protocol is that all intermediate values of the computation seen by the players are uniformly distributed. At each stage, the players obtain random shares v_1 and v_2 such

⁵We stress that in actuality, this protocol does *not* fulfill the security requirements in the general case since A learns the structure of the circuit computing f (see details in Appendix B). This problem is further compounded due to the fact that if f is public and we are in fact computing $f(x, y)$, then B is able to alter f which cannot be done in the ideal model. However, in our scenario where f is known *and* the parties are semi-honest, this protocol fulfills all requirements.

that their sum equals an appropriate intermediate value. Efficiency is achieved by having the parties do most of the computation independently. Recall that there is a known upper bound on the size of the union of the databases, and that the attribute-value names are public.

Solution Outline. The “most difficult” step in privately computing $ID3_\delta$ reduces to oblivious evaluation of the $x \ln x$ function (Section 5.1). (A private protocol for this task is presented separately in Section 6.) Next, we show how given a protocol for computing $x \ln x$, we can privately find the next attribute in the decision tree (Section 5.2). Finally, we describe how the other steps of the $ID3_\delta$ algorithm are privately computed and show the complete private protocol for computing $ID3_\delta$ (Section 5.3).

5.1 A Closer Look at $ID3_\delta$

The part of $ID3_\delta$ which is hardest to implement in a private manner is step 3(a) (see Figure 1). In this step the two parties must find the attribute A that best classifies the transactions T in the database, namely the attribute that provides the maximum information gain. This step can be stated as:

Find the attribute A which minimizes the conditional information of T given A , $H_C(T|A)$.

Examine $H_C(T|A)$ for an attribute A with m possible values $a_1, \dots, a_m$, and a class attribute C with ℓ possible values $c_1, \dots, c_\ell$.

$$\begin{aligned}
H_C(T|A) &= \sum_{j=1}^m \frac{|T(a_j)|}{|T|} H_C(T(a_j)) \\
&= \frac{1}{|T|} \sum_{j=1}^m |T(a_j)| \sum_{i=1}^{\ell} -\frac{|T(a_j, c_i)|}{|T(a_j)|} \cdot \log\left(\frac{|T(a_j, c_i)|}{|T(a_j)|}\right) \\
&= \frac{1}{|T|} \left(-\sum_{j=1}^m \sum_{i=1}^{\ell} |T(a_j, c_i)| \log(|T(a_j, c_i)|) + \sum_{j=1}^m |T(a_j)| \log(|T(a_j)|) \right) \quad (3)
\end{aligned}$$

Note that since the algorithm is only interested in finding the attribute A which minimizes $H_C(T|A)$, the coefficient $1/|T|$ can be ignored. Also, natural logarithms can be used instead of logarithms to the base 2.

The database is a union of two databases, D_1 which is known to P_1 and D_2 which is known to P_2 . The number of transactions for which attribute A has value a_j can therefore be written as $|T(a_j)| = |T_1(a_j)| + |T_2(a_j)|$, where $|T_b(a_j)|$ is the number of transactions with attribute A set to a_j in database D_b (likewise $|T(a_j, c_i)|$ is the number of transaction with $A = a_j$ and the class attribute set to c_i). The values $|T_1(a_j)|$ and $|T_1(a_j, c_i)|$ can be computed by party P_1 independently, and the same holds for P_2 . Therefore the expressions that should be compared can be written as a sum of expressions of the form

$$(v_1 + v_2) \cdot \ln(v_1 + v_2),$$

where v_1 is known to P_1 and v_2 is known to P_2 . (e.g., $v_1 = |T_1(a_j)|$, $v_2 = |T_2(a_j)|$). The main task is, therefore, to privately compute $x \ln x$ and a protocol for this task is described in Section 6. The protocol receives private inputs x_1 and x_2 such that $x_1 + x_2 = x$ and outputs random shares of $x \ln x$ (see Figure 3 in Section 6 for an exact definition of the protocol).

5.2 Finding the Attribute with Maximum Gain

Given the above protocol for privately computing shares of $x \ln x$, the attribute with the maximum information gain can be determined. This is done in two stages: first, the parties obtain shares of $H_C(T|A) \cdot |T| \cdot \ln 2$ for all attributes A and second, the shares are input into a very small circuit which outputs the appropriate attribute. In this section we refer to a field $\mathcal{F}$ which is defined so that $|\mathcal{F}| > H_C(T|A) \cdot |T| \cdot \ln 2$.

Stage 1 (computing shares): For every attribute A , every attribute-value $a_j \in A$ and every class $c_i \in C$, P_1 and P_2 use the $x \ln x$ protocol in order to obtain random shares $w_{A,1}(a_j)$, $w_{A,2}(a_j)$, $w_{A,1}(a_j, c_i)$ and $w_{A,2}(a_j, c_i) \in_R \mathcal{F}$ such that

$$\begin{aligned} w_{A,1}(a_j) + w_{A,2}(a_j) &= |T(a_j)| \cdot \ln(|T(a_j)|) \mod |\mathcal{F}| \\ w_{A,1}(a_j, c_i) + w_{A,2}(a_j, c_i) &= |T(a_j, c_i)| \cdot \ln(|T(a_j, c_i)|) \mod |\mathcal{F}| \end{aligned}$$

Now, define $\hat{H}_C(T|A) \stackrel{\text{def}}{=} H_C(T|A) \cdot |T| \cdot \ln 2$. Then,

$$\hat{H}_C(T|A) = - \sum_{j=1}^m \sum_{i=1}^{\ell} |T(a_j, c_i)| \cdot \ln(|T(a_j, c_i)|) + \sum_{j=1}^m |T(a_j)| \cdot \ln(|T(a_j)|)$$

Then given the above shares, P_1 (and likewise P_2) computes his share in $\hat{H}_C(T|A)$ as follows:

$$S_{A,1} = - \sum_{j=1}^m \sum_{i=1}^{\ell} w_{A,1}(a_j, c_i) + \sum_{j=1}^m w_{A,1}(a_j) \mod |\mathcal{F}|$$

It is clear that $S_{A,1} + S_{A,2} = \hat{H}_C(T|A) \mod |\mathcal{F}|$ and we therefore have that for every attribute A , P_1 and P_2 obtain shares in $\hat{H}_C(T|A)$ (this last step involves local computation only).

Stage 2 (finding the attribute): It remains to find the attribute minimizing $\hat{H}_C(T|A)$ (and therefore $H_C(T|A)$). This is done via a Yao circuit evaluation [21]. We note that since $\hat{H}_C(T|A) < |\mathcal{F}|$, it holds that either $S_{A,1} + S_{A,2} = \hat{H}_C(T|A)$ or $S_{A,1} + S_{A,2} = \hat{H}_C(T|A) + |\mathcal{F}|$. The parties run an oblivious evaluation of a circuit with the following functionality. The circuit input is the shares of both parties for each $\hat{H}_C(T|A)$. The circuit first computes each $\hat{H}_C(T|A)$ (by subtracting $|\mathcal{F}|$ if the sum is larger than $|\mathcal{F}| - 1$ or leaving it otherwise), and then compares the results to find the smallest among them. The circuit thus outputs the attribute A for which $\hat{H}_C(T|A)$ is minimum. This circuit has $2|R|$ inputs of size $\log |\mathcal{F}|$ and its size is $O(|R| \log |\mathcal{F}|)$. Note that $|R| \log |\mathcal{F}|$ is a small number and thus this circuit evaluation is efficient.

Privacy: Stage 1 is clearly private as it involves many invocations of a private protocol that outputs random shares, followed by a local computation. Stage 2 is also private as it involves a single invocation of Yao's oblivious circuit evaluation and nothing more. By the composition theorem (Theorem 2), we have that the overall protocol is trivially private.

Note the efficiency achieved above. Each party has to compute the same set of values $|T(a_j, c_i)|$ as it computes in an individual computation of ID3. For each of these values it engages in the $x \ln x$ protocol. (We stress that the number of values here does not depend on the number of transactions, but rather on the number of different possible values for each attribute, which is usually smaller by orders of magnitude.) It sums the results of all these protocols together, and engages in an oblivious evaluation of a circuit whose size is linear in the number of attributes.

5.3 The Private ID_{3 δ} Protocol

In the previous subsection we showed how each node can be privately computed. The complete protocol for privately computing ID_{3 δ} can be seen in Figure 2. The steps of the protocol correspond to those in the original algorithm (see Figure 1).

Privacy-Preserving Protocol for ID₃:

Step 1: *If R is empty, return a leaf-node with the class value of the majority of the transactions in T .*

Since the set of attributes is known to both parties, they both publicly know if R is empty. If yes, the parties do an oblivious evaluation of a circuit whose inputs are the values $\langle |T_1(c_1)|, \dots, |T_1(c_\ell)| \rangle$ and $\langle |T_2(c_1)|, \dots, |T_2(c_\ell)| \rangle$, and whose output is i such $|T_1(c_i)| + |T_2(c_i)|$ is maximal. The size of this circuit is linear in ℓ and in $\log(|T|)$.

Step 2: *If all the transactions in T have the same class c , return a leaf-node with the value c .*

In order to compute this step privately, we must determine whether both parties remain with the same single class or not. We define a fixed symbol $-$ symbolizing the fact that a party has more than one remaining class. A party's input to this step is then $-$, or c_i if it is its one remaining class. All that remains to do is check *equality* of the two inputs. The value causing the equality can then be publicly announced as c_i (halting the tree on this path) or $-$ (to continue growing the tree from the current point).

The equality check can be executed in one of two ways: (1) Using the “comparing information without leaking it” protocols of Fagin, Naor, and Winkler [9]. This solution requires the execution of $\log(\ell + 1)$ oblivious transfers. (2) Using a protocol suggested in [16] and which involves the oblivious evaluation of linear polynomials. The overhead of this solution is $O(1)$ oblivious transfers, using the oblivious polynomial evaluation protocol of [7].

Step 3: *(a) Determine the attribute that best classifies the transactions in T , let it be A .*

For every value a_j of every attribute A , and for every value c_i of the class attribute C , the two parties run the $x \ln x$ protocol of Section 6 for $T(a_j)$ and $T(a_j, c_i)$. They then continue as described in Section 5.2 by computing independent additions and inputting the results into a small circuit computing the attribute with the highest information gain. Finally, they perform an oblivious evaluation of the circuit with the result being the attribute A . This is public knowledge as it becomes part of the output.

(b,c) Recursively call ID_{3 δ} for the remaining attributes on the transaction sets $T(a_1), \dots, T(a_m)$ (where $a_1, \dots, a_m$ are the values of attribute A).

The result of 3(a) and the attribute values of A are public and therefore both parties can individually partition the database and prepare their input for the recursive calls.

Figure 2: Protocol for Privately Computing ID_{3 δ}

Although each individual step of the complete protocol has been shown to be private, we must show that the composition is also private. Recall that the composition theorem (Theorem 2) only states that if the *oracle-aided* protocol is private, then so too is the protocol for which we use private protocols instead of oracles. Here we prove that the oracle-aided ID_{3 δ} protocol is private.

The central issue in the proof involves showing that despite the fact that the control flow *depends* on the input (and is not predetermined), a simulator can exactly predict the control flow of the protocol. This is non-trivial and in fact, as we remark below, were we to switch Steps (1) and (2) in the protocol (as the algorithm is in fact presented by Mitchell in [15]) the protocol would no longer be private. Formally, of course, we show how the simulator generates a party's view based solely on the input and output.

Theorem 3 *The protocol for computing ID_{3 δ} is private.*

Proof: In this proof the simulator is described in generic terms as it is identical for P_1 and P_2 . Furthermore, we skip details which are obvious (more exact descriptions of simulators are given in the proofs of Section 6). Recall that the simulator is given the output decision tree.

We need to show that any information learned by the computation can be learned directly from the input and output. This is done by showing how the views can be correctly simulated based solely on the input and output. Recall that the computation of the tree is recursive beginning at the root. For each node, a “splitting” class is chosen (due to it having the highest information gain) developing the tree to the next level. Any implementation defines the order of developing the tree and this order is used by the simulator to write the messages received in the correct order. Therefore according to this order, at any given step the computation is based on finding the highest information gain for a *known* node (for the proof we ignore optimizations which find the gain for more than one node in parallel). We now describe the simulator for each node (the simulation order is according to the defined order of traversal).

We differentiate between two cases: (1) a given node is a leaf node and (2) a given node is not a leaf.

1. *The Current Node in the Computation is a Leaf-Node:* The simulator checks, by looking at the input, if the set of attributes R at this point is empty or not. If it is *not* empty (this can be deduced from the tree and the attribute-list which is public), then the computation proceeds to Step (2). In this case, the simulator writes that the oracle-answer from the equality call in Step (2) is *equal* (or else it would not be a leaf). On the other hand, if the list of attributes *is* empty, the computation is executed in Step (1) and the simulator writes the output of the majority evaluation to be the class appearing in the leaf.
2. *The Current Node in the Computation is not a Leaf-Node:* In this case Step (1) is skipped and the oracle-answer of Step (2) must be *not-equal*; this is therefore what the simulator writes. The computation then proceeds to Step (3) which involves many invocations of the $x \ln x$ protocol, returning values uniformly distributed in $\mathcal{F}$. Therefore, the simulator simply chooses the correct number of random values (based on the public list of attribute names, values and class values) and writes them. The next step of the algorithm is a local computation (not included in the view) and an oblivious circuit evaluation. The simulator simply looks to see which class is written in the tree at this node and writes the class name as the output from the circuit evaluation.

We have therefore shown that there exist simulators S_1 and S_2 that given P_1 and P_2 ’s respective inputs and the output decision tree, output strings which are indistinguishable from P_1 and P_2 ’s views in the real computation. This completes the proof. ■

Remark. It is both surprising and interesting to note that if Steps (1) and (2) of the protocol are switched (as the algorithm is in fact presented in [15]), then it is no longer private. This is due to the equality evaluation in Step (2), which may leak information about the other party’s input. Consider the case of a computation in which at a certain point the list of attributes is *empty* and P_1 has only one class c left in his remaining transactions. The output of the tree at this point is a leaf with a class, assume that the class is c . From the output it is impossible for P_1 to know if P_2 ’s transactions also have only one remaining class or if the result is because the majority of both together is c . The majority circuit of Step (1) covers both cases and therefore does not reveal this information. However, if P_1 and P_2 first execute the equality evaluation, this information is revealed.

5.4 Complexity

The complexity (measuring both communication and computational complexity) for each node is as follows (recall that R denotes the set of attributes and T the set of transactions):

- The $x \ln x$ protocol is repeated $m(\ell+1)$ times for each attribute where m and ℓ are the number of attribute and class values respectively (see Equation (3)). For all $|R|$ attributes we thus have $O(m \cdot \ell \cdot |R|)$ invocations of the $x \ln x$ protocol. The complexity of the $x \ln x$ protocol can be found in Section 6.3. In short, the computational overhead of the $x \ln x$ protocol is dominated by $O(\log |T|)$ oblivious transfers and the bandwidth is $O(k \cdot \log |T| \cdot |S|)$ bits, where k is a parameter depending logarithmically on δ that determines the accuracy of the $x \ln x$ approximation and $|S|$ is the length of the key for a pseudorandom function (say 128 bits).
- As we have mentioned, the size of the circuit computing the attribute with the minimum conditional entropy is $O(|R| \log |\mathcal{F}|)$ where $|\mathcal{F}| = O(|T|)$.⁶ The *bandwidth* involved in sending the garbled circuit is thus $O(|R| \log |T| \cdot |S|)$ where $|S|$ is the length of the key for a pseudorandom function. (This factor is explained in the paragraph titled “overhead” in Appendix B.)

The *computational complexity* of the above circuit evaluation involves $|R| \log |\mathcal{F}| = O(|R| \log |T|)$ oblivious transfers (one for each bit of the circuit input) and $O(|R| \log |T|)$ pseudorandom function evaluations.

- The number of *rounds* needed for each node is constant (the $x \ln x$ protocol also requires only a constant number of rounds, see Section 6).

The overhead of the $x \ln x$ invocations far outweighs the circuit evaluation that completes the computation. We thus consider only these invocations in the overall complexity. The analysis is completed by multiplying the above complexity by the number of nodes in the resulting decision tree (expected to be quite small).⁷ We note that by computing nodes on the same level of the tree in parallel, the number of rounds of communication can be reduced to the order of the *depth* of the tree (which is bounded by $|R|$ but is expected to be far smaller).

In Section 7 we analyze the complexity for concrete parameter values of a large, example database. We further compare the complexity of our protocol to generic protocols for privately computing ID3.

6 A Protocol for Computing $x \ln x$

This section describes an efficient protocol for privately computing the $x \ln x$ function, as defined in Figure 3.

There are several difficulties in the design of such a protocol. Firstly, it is not clear how to obviously compute the natural logarithm efficiently. Furthermore, the protocol must multiply two values together. An initial idea is to use Yao’s generic two party circuit evaluation protocol [21] and construct a multiplication circuit. However, the size of this circuit is of the order of the *multiplication of the sizes of its inputs*. This subprotocol is to be repeated many times throughout

⁶We note that the size of the field $\mathcal{F}$ needed for the $x \ln x$ protocol is actually larger than that required for this part of the protocol. As is described in Section 6, $\log |\mathcal{F}| = O(k \log |T|)$ where k is a parameter depending on δ as described above. However, $k \approx 12$ provides high accuracy and therefore this does not make a significant difference.

⁷Note that the overhead is actually even smaller since the effective number of attributes in a node of depth d' is $|R| - d'$. Since most nodes are at lower levels of the tree and since this is a multiplicative factor in the expression of the overhead, the overhead is decreased substantially. The effective value of $|R|$ for the overall overhead can be reduced to about $|R|$ minus the depth of the resulting decision tree.

- **Input:** P_1 's input is a value v_1 ; P_2 's input is v_2 .
- **Auxiliary input:** A large enough field $\mathcal{F}$, the size of which will be discussed later.
- **Output:** P_1 obtains $w_1 \in \mathcal{F}$ and P_2 obtains $w_2 \in \mathcal{F}$ such that:
 1. $w_1 + w_2 = (v_1 + v_2) \cdot \ln(v_1 + v_2) \bmod |\mathcal{F}|$
 2. w_1 and w_2 are uniformly distributed in $\mathcal{F}$ when viewed independently of one another.

Figure 3: Definition of the $x \ln x$ protocol.

the complete ID3_δ protocol and its efficiency is, therefore, crucial. (See Appendix 7 for an analysis which demonstrates that the overhead of using the generic protocol is larger by an order of magnitude than that of using our solution.)

The solution requires a linear size circuit and a small number of simple oblivious evaluation protocols. The problem is divided into two parts: First it is shown how to compute shares of $\ln x$ from shares of x . Secondly, we show how to obtain shares of the product $x \ln x$ given separate shares of x and $\ln x$.

6.1 Computing Shares of $\ln x$

We now show how to compute random shares u_1 and u_2 such that $u_1 + u_2 = \ln x$. The starting point for the solution is the Taylor series of the natural logarithm, namely:

$$\ln(1 + \varepsilon) = \sum_{i=1}^{\infty} \frac{(-1)^{i-1} \varepsilon^i}{i} = \varepsilon - \frac{\varepsilon^2}{2} + \frac{\varepsilon^3}{3} - \frac{\varepsilon^4}{4} + \dots \quad \text{for } -1 < \varepsilon < 1$$

It is easy to verify that the error for a partial evaluation of the series is as follows:

$$\left| \ln(1 + \varepsilon) - \sum_{i=1}^k \frac{(-1)^{i-1} \varepsilon^i}{i} \right| < \frac{|\varepsilon|^{k+1}}{k+1} \cdot \frac{1}{1 - |\varepsilon|} \quad (4)$$

Thus, the error shrinks exponentially as k grows (see Section 6.4 for an analysis of the cumulative effect of this error in computing ID3_δ).

Now, given an input x , let 2^n be the power of 2 which is closest to x (in the ID3_δ application, note that $n < \log |T|$). Therefore, $x = 2^n(1 + \varepsilon)$ where $-1/2 \leq \varepsilon \leq 1/2$. Consequently,

$$\ln(x) = \ln(2^n(1 + \varepsilon)) = n \ln 2 + \varepsilon - \frac{\varepsilon^2}{2} + \frac{\varepsilon^3}{3} - \frac{\varepsilon^4}{4} + \dots$$

Our aim is to compute this Taylor series to the k 'th place. Let N be a predetermined (public) upper-bound on the value of n ($N > n$ always). Now, we use a small circuit that receives v_1 and v_2 as input (the value of N is hardwired into it) and outputs shares of the following values

- $2^N \cdot n \ln 2$ (for computing the first element in the series of $\ln x$)
- $\varepsilon \cdot 2^N$ (for computing the remainder of the series).

This circuit is easily constructed: notice that $\varepsilon \cdot 2^n = x - 2^n$, where n can be determined by looking at the two most significant bits of x , and $\varepsilon \cdot 2^N$ is obtained simply by shifting the result by $N - n$ bits to the left. The possible values of $2^N n \ln 2$ are hardwired into the circuit. As we have described, random shares are obtained by having one of the parties input random values $\alpha_1, \beta_1 \in_R \mathcal{F}$ into the

circuit and having the circuit output $\alpha_2 = \varepsilon \cdot 2^N - \alpha_1$ and $\beta_2 = 2^N \cdot n \ln 2 - \beta_1$ to the other party. The parties therefore have shares α_1, β_1 and α_2, β_2 such that

$$\alpha_1 + \alpha_2 = \varepsilon 2^N \quad \text{and} \quad \beta_1 + \beta_2 = 2^N n \ln 2$$

The second stage of the protocol involves computing shares of the Taylor series approximation. In fact, it computes shares of

$$\text{lcm}(2, \dots, k) \cdot 2^N \left(n \ln 2 + \varepsilon - \frac{\varepsilon^2}{2} + \frac{\varepsilon^3}{3} - \dots - \frac{\varepsilon^k}{k} \right) \approx \text{lcm}(2, \dots, k) 2^N \cdot \ln x \quad (5)$$

(where $\text{lcm}(2, \dots, k)$ is the lowest common multiple of $\{2, \dots, k\}$, and we multiply by it to ensure that there are no fractions). In order to do this P_1 defines the following polynomial:

$$Q(x) = \text{lcm}(2, \dots, k) \cdot \sum_{i=1}^k \frac{(-1)^{i-1}}{2^{N(i-1)}} \frac{(\alpha_1 + x)^i}{i} - w_1$$

where $w_1 \in_R \mathcal{F}$ is randomly chosen. It is easy to see that

$$w_2 \stackrel{\text{def}}{=} Q(\alpha_2) = \text{lcm}(2, \dots, k) \cdot 2^N \cdot \left(\sum_{i=1}^k \frac{(-1)^{i-1} \varepsilon^i}{i} \right) - w_1$$

Therefore after a single oblivious polynomial evaluation of the k -degree polynomial $Q(\cdot)$, P_1 and P_2 obtain random shares w_1 and w_2 to the approximation in Equation (5). Namely P_1 defines $u_1 = w_1 + \text{lcm}(2, \dots, k) \beta_1$ and likewise P_2 . We conclude that

$$u_1 + u_2 \approx \text{lcm}(2, \dots, k) 2^N \cdot \ln x$$

This equation is accurate up to an approximation error which depends on k , and the shares are random as required. Since N and k are known to both parties, the additional multiplicative factor of $2^N \cdot \text{lcm}(2, \dots, k)$ is public and can be removed at the very end. Notice that *all* the values in the computation are integers (except for $2^N n \ln 2$ which is given as the closest integer).

The size of the field $\mathcal{F}$. It is necessary that the field be chosen large enough so that the initial inputs in each evaluation and the final output be between 0 and $|\mathcal{F}| - 1$. Notice that all computation is based on $\varepsilon 2^N$. This value is raised to powers up to k and multiplied by $\text{lcm}(2, \dots, k)$. Therefore a field of size 2^{Nk+2k} is clearly large enough, and requires $(N+2)k$ bits for representation.

We now summarize the $\ln x$ protocol (recall that N is a public upper bound on $\log |T|$):

Protocol 1 (Protocol $\ln x$)

1. P_1 and P_2 input their shares v_1 and v_2 into an oblivious evaluation protocol for a circuit outputting: (1) Random shares α_1 and α_2 of $\varepsilon 2^N$ (i.e. $\alpha_1 + \alpha_2 = \varepsilon 2^N \bmod |\mathcal{F}|$) and (2) Random shares β_1, β_2 such that $\beta_1 + \beta_2 = 2^N \cdot n \ln 2$. We define that P_1 constructs the circuit and sends it to P_2 (this has no importance except to fix the order of operations).
2. P_1 chooses $w_1 \in_R \mathcal{F}$ and defines the following polynomial

$$Q(x) = \text{lcm}(2, \dots, k) \cdot \sum_{i=1}^k \frac{(-1)^{i-1}}{2^{N(i-1)}} \frac{(\alpha_1 + x)^i}{i} - w_1$$

3. P_1 and P_2 then execute an oblivious polynomial evaluation with P_1 inputting $Q(\cdot)$ and P_2 inputting α_2 , in which P_2 obtains $w_2 = Q(\alpha_2)$.
4. P_1 and P_2 define $u_1 = \text{lcm}(2, \dots, k)\beta_1 + w_1$ and $u_2 = \text{lcm}(2, \dots, k)\beta_2 + w_2$ respectively. We have that $u_1 + u_2 \approx 2^N \text{lcm}(2, \dots, k) \cdot \ln x$

Note that in order to obtain shares of just $\ln x$ (which we *do not* wish to do here, but might be required for another setting), each party simply divides its share by $2^N \cdot \text{lcm}(2, \dots, k)$.

We now prove that the $\ln x$ protocol is correct and secure. We prove correctness by showing that the field and intermediate values are such that the output shares uniquely define the result. Privacy is shown by showing simulators for the views; these are easily defined as we use the polynomial evaluation as an oracle-functionality.

Before beginning the proof, we introduce notation for measuring the accuracy of the approximation. That is, we say that $\tilde{x}$ is a Δ -approximation of x if $|x - \tilde{x}| \leq \Delta$.

Proposition 4 *Protocol 1 constitutes a private protocol for computing random shares of a $\frac{c}{2^k(k+1)}$ -approximation of $c \cdot \ln x$ in $\mathcal{F}$, where $c = 2^N \cdot \text{lcm}(2, \dots, k)$.*

Proof: We show that the protocol correctly computes shares of an approximation of $c \ln x$. In order to do this, we must show that the computation over $\mathcal{F}$ results in a correct result over the reals. We first note that *all* the intermediate values are integers. In particular, $\varepsilon 2^n$ equals $x - 2^n$ and is therefore an integer as is $\varepsilon 2^N$ (since $N > n$). Furthermore, every division by i ($2 \leq i \leq k$) is counteracted by a multiplication by $\text{lcm}(2, \dots, k)$. The only exception is $2^N n \ln 2$. However, this is taken care of by having the original circuit output the closest integer to $2^N n \ln 2$. (Although the rounding to the closest integer introduces an additional approximation error, it can be made negligible compared to the approximation error of the Taylor series.)

Secondly, the field $\mathcal{F}$ is defined to be large enough so that all intermediate values (i.e. the sum of shares) and the final output (as a real number times $2^N \cdot \text{lcm}(2, \dots, k)$) are between 0 and $|\mathcal{F}| - 1$. Therefore the two shares uniquely identify the result, which equals the sum (over the integers) of the two random shares if it is less than $|\mathcal{F}| - 1$, or the sum minus $|\mathcal{F}|$ otherwise.

Finally, we show that the accuracy of the approximation is as desired. As we have mentioned in Equation (4), the $\ln(1 + \varepsilon)$ error is bounded by $\frac{|\varepsilon|^{k+1}}{k+1} \frac{1}{1-|\varepsilon|}$. Since $-\frac{1}{2} \leq \varepsilon \leq \frac{1}{2}$, we have that this error rate is maximum at $|\varepsilon| = \frac{1}{2}$. We therefore have that $\left| \tilde{\ln}(1 + \varepsilon) - \ln(1 + \varepsilon) \right| \leq \frac{1}{2^k(k+1)}$, where $\tilde{\ln}(1 + \varepsilon)$ denotes the approximation of our protocol. Now, $c \ln x = cn \ln 2 + c \ln(1 + \varepsilon)$ and therefore by adding $cn \ln 2$ to both $c \ln(1 + \varepsilon)$ and $c \tilde{\ln}(1 + \varepsilon)$ we have that this has no effect on the error. However, we actually add an *approximation* of $cn \ln 2$ to $c \tilde{\ln}(1 + \varepsilon)$ in the protocol. Nevertheless, the error of this approximation can be made to be much smaller than $\frac{c}{2^k(k+1)}$. This is because the approximation of $n \ln 2$ is hardwired into the circuit and thus the error can be made as small as desired. Therefore, the total error of $c \tilde{\ln} x$ is $\frac{c}{2^k(k+1)}$, which means that the effective error of the approximation of $\ln x$ is only $\frac{1}{2^k(k+1)}$.

Privacy: The proof of privacy is based on Theorem 2 (the composition theorem) by considering the circuit and polynomial evaluations as oracle-functionalities. We show simulators S_1 and S_2 for simulating the necessary views. That is, a simulator S_1 is given (v_1, u_1) and must generate an output string so that $\{(S_1(v_1, u_1), u_2)\}$ is indistinguishable from $\{(\text{view}_1^{\ln x}(v_1, v_2), \text{output}_2^{\ln x}(v_1, v_2))\}$. (Note that u_1 and u_2 here denote the outputs as required by the $\ln x$ problem definition and not the results of the protocol execution.)

Simulator S_1 : The algorithm for the simulator S_1 is trivial as P_1 receives no output from any of the oracle-functionalities. However, P_1 's random string implicitly defines his output and we must therefore be careful that it is consistent with u_1 (this will ensure that the joint distribution of S_1 's output and u_2 will be as required). Therefore, S_1 chooses 2 random elements $\alpha_1, \beta_1 \in_R \mathcal{F}$ and fixes w_1 (his share from the polynomial evaluation) such that $u_1 = \text{lcm}(2, \dots, k) \cdot \beta_1 + w_1$. S_1 then writes $(v_1, r = (\alpha_1, \beta_1, w_1))$ (there are no messages received by P_1 during the protocol - the oracle-functionalities output to P_2 only - and therefore this is P_1 's entire view).

It is easily seen that S_1 's output is distributed identically to $\text{view}_1^{\ln x}(v_1, v_2)$ (this is true with respect to the oracle-aided protocol but by plugging in the circuit and polynomial evaluation simulators the output will be computationally indistinguishable). Furthermore, by the functionality definition, the sum of P_1 and P_2 's outputs must be $2^N \text{lcm}(2, \dots, k) \ln x$. Since (β_1, w_1) were chosen so that they define u_1 (as given to S_1), we have that the joint distribution of $\{((v_1, (\alpha_1, \beta_1, w_1)), u_2)\}$ is indistinguishable from P_1 's view and P_2 's output from the protocol.

Simulator S_2 : In order to define S_2 notice that P_2 's output from each oracle-functionality (by definition) is a uniformly distributed element in $\mathcal{F}$. Therefore, S_2 simply chooses $\alpha_2, \beta_2 \in_R \mathcal{F}$ and fixes w_2 such that $u_2 = \text{lcm}(2, \dots, k) \cdot \beta_2 + w_2$. S_2 then writes $(v_2, -, (\alpha_2, \beta_2), w_2)$ and this completes the simulator. Once again, S_2 's view is consistent with u_2 and this ensures that the joint distribution of $\{(u_1, S_2(v_2, u_2))\}$ is indistinguishable from P_1 's output and P_2 's view. ■

6.2 Computing Shares of $x \ln x$

We begin by briefly describing a simple multiplication protocol that on private inputs a_1 and a_2 outputs random shares b_1 and b_2 (in some finite field $\mathcal{F}$) such that $b_1 + b_2 = a_1 \cdot a_2$. The protocol is very simple and is based on a single oblivious evaluation of a linear polynomial.

Protocol 2 (Protocol $\text{Mult}(a_1, a_2)$)

1. P_1 chooses a random value $b_1 \in \mathcal{F}$ and defines the linear polynomial $Q(x) = a_1 x - b_1$.
2. P_1 and P_2 engage in an oblivious evaluation of Q , in which P_2 obtains $b_2 = Q(a_2) = a_1 \cdot a_2 - b_1$.
3. The respective outputs of P_1 and P_2 are defined as b_1 and b_2 , giving us that $b_1 + b_2 = a_1 \cdot a_2$.

The correctness of the protocol (i.e., that b_1 and b_2 are uniformly distributed in $\mathcal{F}$ and sum up to $a_1 \cdot a_2$) is obvious from the definition of Q . Furthermore, the privacy is immediate as the protocol consists entirely of a single evaluation of a private protocol (i.e., a single oblivious polynomial evaluation). We thus have the following proposition:

Proposition 5 *Protocol 2 constitutes a private protocol for computing Mult as defined above.*

As discussed in Section 4.2, oblivious evaluation of linear polynomials can be computed using a constant number of exponentiations.

We are now ready to present the complete $x \ln x$ protocol (recall that P_1 and P_2 's respective inputs are v_1 and v_2 where $v_1 + v_2 = x$):

Protocol 3 (Protocol $x \ln x$)

1. P_1 and P_2 run Protocol 1 for privately computing shares of $\ln x$ and obtain random shares u_1 and u_2 such that $u_1 + u_2 = \ln x$.
2. P_1 and P_2 use two invocations of Protocol 2 in order to obtain shares of $u_1 \cdot v_2$ and $u_2 \cdot v_1$.

3. P_1 (resp., P_2) then defines his output w_1 (resp., w_2) to be the sum of the two *Mult* shares and $u_1 \cdot v_1$ (resp., $u_2 \cdot v_2$).
4. We have that $w_1 + w_2 = u_1 v_1 + u_1 v_2 + u_2 v_1 + u_2 v_2 = (u_1 + u_2)(v_1 + v_2) = x \ln x$ as required.

Theorem 6 *Protocol 3 is a protocol for privately computing random shares of $x \ln x$.*

Proof: We show a simulator S_1 for simulating $\text{view}_1^{x \ln x}(v_1, v_2)$ such that $\{S_1(v_1, w_1), w_2\}$ is indistinguishable from $\{(\text{view}_1^{x \ln x}(v_1, v_2), \text{output}_2^{x \ln x}(v_1, v_2))\}$ (the simulator S_2 is identical). S_1 receives as input (v_1, w_1) (input/output) of P_1 . Recall that the view of a party is defined by $(x, r, m_1, m_2, \dots)$ where x is the input, r the private coin tosses and m_i the i 'th message received.

S_1 , the simulator for P_1 's view, upon input (v_1, w_1) first chooses $r_1, s_1 \in_R \mathcal{F}$ (these values simulate the shares of $u_1 \cdot v_2$ and $u_2 \cdot v_1$). S_1 then fixes u_1 so that $u_1 v_1 + r_1 + s_1 = w_1$ (u_1 simulates the output from the $\ln x$ protocol), and writes $(v_1, -, u_1, r_1, s_1)$. From the protocol definition, it is immediate that S_1 's output is identically distributed to $\text{view}_1^{x \ln x}(v_1, v_2)$. Furthermore, since the simulated view output by S_1 implicitly defines w_1 and by definition $w_1 + w_2 = x \ln x$, we have that $\{S_1(v_1, w_1), w_2\}$ is indistinguishable from the joint distribution of P_1 's view and P_2 's output.

The simulators clearly output a view indistinguishable from the real one. Basing ourselves on the composition theorem (Theorem 2), this completes the proof. ■

6.3 Complexity

The $\ln x$ Protocol (Protocol 1):

1. Step 1 of the protocol (computing random shares $\alpha_1, \alpha_2, \beta_1$ and β_2) involves an oblivious evaluation of a simple (linear) circuit. The *bandwidth* involved in sending the garbled circuit is $O(\log |\mathcal{F}| |S|) = O(k \log |T| \cdot |S|)$ communication bits where $|S|$ is the length of the key for a pseudorandom function. (This is explained in Appendix B.)

The *computational complexity* is dominated by the $O(\log |T|)$ oblivious transfers required for every bit of the circuit input.

2. Steps 2 and 3 of the protocol (computing the Taylor series) involve the oblivious evaluation of a polynomial of degree k over the field $\mathcal{F}$. The oblivious evaluation can be computed using $O(k)$ exponentiations and $O(k)$ messages of total length $O(k|E|)$ where $|E|$ is the length of an element in the group in which the oblivious transfers and exponentiations are implemented.

The overall computation overhead of the protocol is thus $O(\max\{\log |T|, k\})$ exponentiations. Since $|T|$ is usually large (e.g. $\log |T| = 20$), and on the other hand k can be set to small values (e.g. $k = 12$, see below), the computational overhead can be defined as $O(\log |T|)$ oblivious transfers. The main communication overhead is incurred by Step 1, and is $O(k \log |T| \cdot |S|)$ bits.

The *Mult* Protocol (Protocol 2): This protocol involves a single linear polynomial evaluation by the players. This can be done with $O(1)$ exponentiations (see Section 4.2) and a constant number of rounds with total bandwidth $O(|E|)$ (where $|E|$ is as defined above).

The $x \ln x$ Protocol (Protocol 3): This step involves one invocation of Protocol 1, and two invocations of Protocol 2. Its overhead is therefore dominated by Protocol 1. We conclude that the overall computational complexity is $O(\log |T|)$ oblivious transfers and that the bandwidth is $O(k \log |T| \cdot |S|)$ bits.

6.4 Choosing the Parameter k

Recall that the parameter k defines the accuracy of the Taylor approximation of the “ $\ln$ ” function. Given δ , we analyze the value of k needed in order to ensure that the defined δ -approximation is correctly estimated. From here on we denote an approximation of the value z by $\widetilde{z}$. The approximation definition of ID3_δ requires that for all A_1, A_2

$$H_C(T|A_1) > H_C(T|A_2) + \delta \Rightarrow \widetilde{H}_C(T|A_1) > \widetilde{H}_C(T|A_2)$$

This is clearly fulfilled if $\left|H_C(T|A_1) - \widetilde{H}_C(T|A_1)\right| < \frac{\delta}{2}$ and $\left|H_C(T|A_2) - \widetilde{H}_C(T|A_2)\right| < \frac{\delta}{2}$

We now bound the difference on each $|\ln x - \widetilde{\ln x}|$ in order that the above condition is fulfilled. By replacing $\log x$ by $\frac{1}{\ln 2}|\ln x - \widetilde{\ln x}|$ in Equation (3) computing $H_C(T|A)$, we obtain a bound on the error of $\left|H_C(T|A_1) - \widetilde{H}_C(T|A_1)\right|$. A straightforward algebraic manipulation gives us that if $\frac{1}{\ln 2}|\ln x - \widetilde{\ln x}| < \frac{\delta}{4}$, then the error is less than $\frac{\delta}{2}$ as required.⁸ By Proposition 4, we have that the $\ln x$ error is bounded by $\frac{1}{2^{k(k+1)}}$ (the multiplicative factor of c is common to all attributes and can therefore be ignored). Therefore, given δ , we set $\frac{1}{2^{k(k+1)}} < \frac{\delta}{4} \cdot \ln 2$ or $k + \log(k+1) > \log \left\lceil \frac{4}{\delta \ln 2} \right\rceil$ (for $\delta = 0.0001$, it is enough to take $k > 12$). Notice that the value of k is *not* dependent on the input database.

7 Protocol Efficiency

A detailed analysis of the complexity of the $x \ln x$ protocol can be found in Section 6.3 and the overall ID3_δ complexity is analyzed in Section 5.4. In this section we demonstrate the efficiency of our protocol with a concrete analysis based on example parameters for an input database. Furthermore, a comparison of the efficiency of our protocol to that of generic solutions is presented.

A Concrete Example. Assume that there are a million transactions $|T| = 2^{20}$, $|R| = 15$ attributes, each attribute has $m = 10$ possible values, the class attribute has $\ell = 4$ values, and $k = 10$ suffices to have the desired accuracy. Say that the depth of the tree is $d = 7$, and that the length of a key for the pseudorandom function is $|S| = 80$ bits.

As is described in Section 5.4 there are $O(m \cdot \ell \cdot |R|) \approx 600$ invocations of the $x \ln x$ protocol for each node and these dominate the overall complexity.

- **Bandwidth:** Each invocation has a communication overhead of $O(k \log |T| \cdot |S|) \approx 16,000$ bits. We conclude that for each node 9.6×10^6 bits (times some small constant factor) are sent. This can be transmitted in a matter of seconds using a fast communication network (e.g. a T1 line with 1.5Mbps bandwidth, or a T3 line with 35Mbps).

⁸The full calculation is as follows:

$$\begin{aligned} & \left|H_C(T|A) - \widetilde{H}_C(T|A)\right| \\ & \leq \frac{1}{|T|} \left(\sum_{j=1}^m \sum_{i=1}^{\ell} |T(a_j, c_i)| \cdot \left| \log |T(a_j, c_i)| - \log |\widetilde{T}(a_j, c_i)| \right| + \sum_{j=1}^m |T(a_j)| \cdot \left| \log |T(a_j)| - \log |\widetilde{T}(a_j)| \right| \right) \\ & < \frac{1}{|T|} \left(\sum_{j=1}^m \sum_{i=1}^{\ell} |T(a_j, c_i)| \cdot \frac{\delta}{4} + \sum_{j=1}^m |T(a_j)| \cdot \frac{\delta}{4} \right) = \frac{1}{|T|} \left(\frac{\delta}{4} \cdot |T| + \frac{\delta}{4} \cdot |T| \right) = \frac{\delta}{2} \end{aligned}$$

- **Computation:** The computation overhead for each $x \ln x$ protocol is $O(\log |T|) \approx 20$ oblivious transfers (and thus exponentiations). We therefore have an order of 12,000 exponentiations and oblivious transfers per node. Assuming that a modern PC can compute 50 exponentiations per second, we conclude that the computation per node can be completed in a matter of minutes.

A Comparison to Generic Solutions. Consider a generic solution for the entire ID3 _{δ} task using Yao’s two party protocol. Such a solution would require $O(|R| \cdot |T| \cdot \log m)$ oblivious transfers (the size of the input). For the above example parameters, we have 60,000,000 overall exponentiations, instead of an order of 12,000 per node. Furthermore, as the number of transactions $|T|$ grows, the gap between the complexities grows rapidly, since the overhead of our protocol is only logarithmic in $|T|$. The size of the circuit sent is also at least $O(|R| \cdot |T| \cdot |S|)$ (a very optimistic estimate) which is once again much larger than in our protocol.

We now consider a *semi-generic* solution for which the protocol is exactly as described in Figure 2. However, a generic (circuit-based) solution is used for the $x \ln x$ protocol instead of the protocol of Section 6. This generic protocol should compute the Taylor series, namely k multiplications in $\mathcal{F}$, with a communication overhead of $O(k \log^2 |\mathcal{F}| |S|) = O(k^3 \log^2 |T| |S|)$ (circuit multiplication is *quadratic* in the length of the inputs). This is larger by a factor of $O(k^2 \log |T|)$ than the overhead of our protocol. As this protocol is repeated $m \cdot \ell \cdot |R|$ times for each node, this is a significant increase in the bandwidth. Specifically, for our example, the bandwidth is $2000 \times 600 = 1,200,000$ times larger. On the other hand, the number of oblivious transfers would remain much the same in both cases. The generic protocol would also require $O(k^3 \log^2 |T|)$ pseudorandom function evaluations but this too is most likely dominated by the complexity of the oblivious transfers.

We note that in the case that the data is fairly random, the output decision tree is likely to be huge and our protocol will not be efficient. However, this is unexpected in real situations.

8 Future Work

An immediate extension of this work is to the case of many (rather than two) parties. A problem which arises is that Yao’s protocol does not extend to the case of many parties. Using other generic protocols for multiparty computation, even on small circuits, can render the solution impractical due to the many number of rounds necessary (each gate is interactively computed). The same problem is true of oblivious polynomial evaluation, a central tool in our construction.

A very important open problem is to design an efficient protocol which is secure even when one of the parties is malicious and does not necessarily act according to the protocol. Other future work includes considering other important data mining algorithms such as Neural Networks and Association Rules Mining.

References

- [1] M. Bellare and S. Micali, *Non-interactive oblivious transfer and applications*, Advances in Cryptology - Crypto ’89, pp. 547-557, 1990.
- [2] M. Ben-Or, S. Goldwasser and A. Wigderson, *Completeness theorems for non cryptographic fault tolerant distributed computation*, 20th STOC, (1988), 1-9.
- [3] D. Boneh and M. Franklin, *Efficient generation of shared RSA keys*, Proc. of Crypto’ 97, LNCS, Vol. 1233, Springer-Verlag, pp. 425-439, 1997.

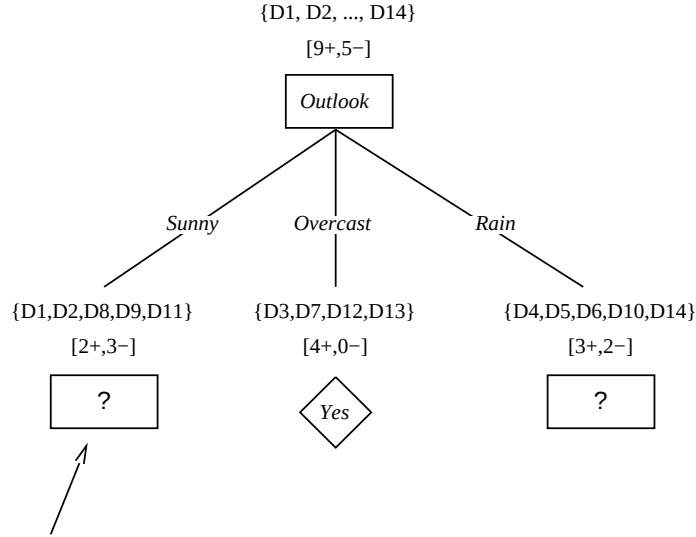
- [4] R. Canetti, *Security and Composition of Multi-party Cryptographic Protocols*, Journal of Cryptology, Vol. 13, No. 1, pages 143–202, 2000.
- [5] D. Chaum, C. Crepeau and I. Damgard, *Multiparty unconditionally secure protocols*, 20th Proc. ACM Symp. on Theory of Computing, (1988), 11-19.
- [6] B. Chor, O. Goldreich, E. Kushilevitz and M. Sudan, *Private Information Retrieval*, 36th FOCS, pp. 41–50, 1995.
- [7] R. Cramer, N. Gilboa, M. Naor, B. Pinkas and G. Poupard, *Oblivious Polynomial Evaluation*, 2000.
- [8] S. Even, O. Goldreich and A. Lempel, *A Randomized Protocol for Signing Contracts*, Communications of the ACM **28**, pp. 637–647, 1985.
- [9] R. Fagin, M. Naor and P. Winkler, *Comparing Information Without Leaking It*, Communications of the ACM, vol 39, May 1996, pp. 77-85.
- [10] J. Feigenbaum, J. Fong, M. Strauss and R. N. Wright, *Secure Multiparty Computation of Approximations*, manuscript, 2000.
- [11] N. Gilboa, *Two Party RSA Key Generation*, Proc of Crypto '99, Lecture Notes in Computer Science, Vol. 1666, Springer-Verlag, pp. 116–129, 1999.
- [12] O. Goldreich, *Secure Multi-Party Computation*, 1998. (Available at <http://philby.ucsd.edu>)
- [13] O. Goldreich, S. Micali and A. Wigderson, *How to Play any Mental Game - A Completeness Theorem for Protocols with Honest Majority*. In 19th ACM Symposium on the Theory of Computing, pp. 218-229, 1987.
- [14] J. Kilian, **Uses of randomness in algorithms and protocols**, MIT Press, 1990.
- [15] T. Mitchell, **Machine Learning**. McGraw Hill, 1997.
- [16] M. Naor and B. Pinkas, *Oblivious Transfer and Polynomial Evaluation*, Proc. of the 31st STOC, Atlanta, GA, pp. 245-254, May 1-4, 1999.
- [17] M. Naor and B. Pinkas, *Efficient Oblivious Transfer Protocols*, to appear in the proceedings of SODA 2001.
- [18] P. Paillier, *Public-Key Cryptosystems Based on Composite Degree Residuosity Classes*. Proc. of Eurocrypt '99, LNCS Vol. 1592, pp. 223–238, 1999.
- [19] J. Ross Quinlan, *Induction of Decision Trees*. Machine Learning 1(1): 81-106(1986)
- [20] M. O. Rabin, *How to exchange secrets by oblivious transfer*, Tech. Memo TR-81, Aiken Computation Laboratory, 1981.
- [21] A.C. Yao, *How to generate and exchange secrets*, Proc. of the 27th IEEE Symp. on Foundations of Computer Science, 1986, pp. 162–167.

A A Decision Tree Example

In this appendix we give an example⁹ of a data set and the resulting decision tree. The aim of the task here is to learn the weather conditions suitable for playing tennis.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

The first attribute chosen in the tree for the above database is *Outlook*. This is seen by a quick computation of the *Gain*. By a quick calculation one can confirm that $Gain(T, Outlook) = 0.246$ which is maximum over the gain of all other attributes. We can see the entropy *gain* calculation for one of the lower nodes in the tree below.



Which attribute should be tested here?

$$S_{\text{sunny}} = \{D1, D2, D8, D9, D11\}$$

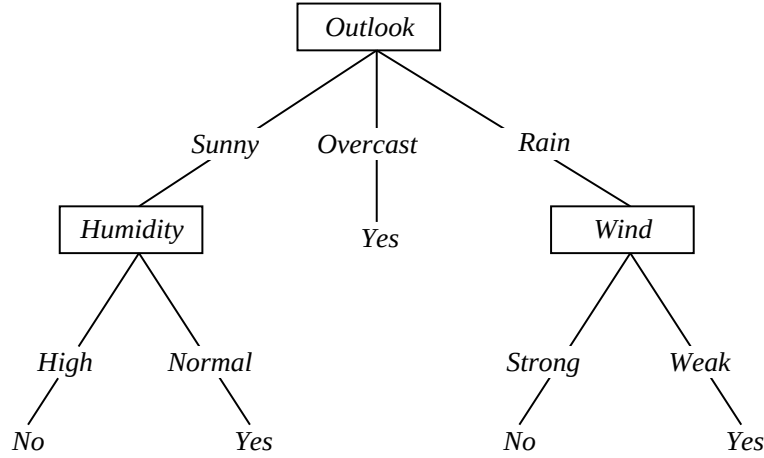
$$Gain(S_{\text{sunny}}, \text{Humidity}) = .970 - (3/5) 0.0 - (2/5) 0.0 = .970$$

$$Gain(S_{\text{sunny}}, \text{Temperature}) = .970 - (2/5) 0.0 - (2/5) 1.0 - (1/5) 0.0 = .570$$

$$Gain(S_{\text{sunny}}, \text{Wind}) = .970 - (2/5) 1.0 - (3/5) .918 = .019$$

⁹The examples are taken from Chapter 3 of Tom Mitchell's book *Machine Learning*, see [15].

It is clear that “Humidity” is the best choice by its *Gain* value. Intuitively, this is logical as it completes the classification in the most concise manner (there is no need for any further tests). Below, we can see the final decision tree.



A new transaction is then classified by traversing the tree according to the attribute/values. For example, the transaction:

(Outlook = Sunny, Temperature = Hot, Humidity = High, Wind = Strong)

is classified as *No* by seeing that it is a sunny and humid day. This example demonstrates ID3’s bias in favor of short hypotheses (at most two out of the four attributes are tested).

Our aim is to output a tree such as this one, such that *nothing* can be learned about the other party’s database that cannot be learned from the output itself.

B Secure Function Evaluation for Two Parties

We describe a secure function evaluation protocol for *two* parties, which is a variant of the protocol of Yao [21] (see also [13]). The protocol is run between two parties, the *Input Owner A* and the *Program Owner B*. The input of *A* is a value x , and the input of *B* is a description of a function f . At the end of the protocol, *A* should learn $f(x)$ (and no other information about f), and *B* should learn nothing about x . (If the function f has two inputs, x which is known to *A* and y which is known to *B*, then *B* hardwires y into the circuit and lets *A* evaluate a circuit computing $f(\cdot, y)$).

Note that in actuality information about the function f is revealed in that *A* sees the structure of the circuit (this structure can be hidden though, if a the function is encoded in a “universal” circuit whose wiring does not depend on the function). However, for our protocol where f is known and y is what must remain secret, no information is revealed. Furthermore, because the players are semi-honest, we know that *B* uses $f(\cdot, y)$ and not some other function.

The protocol is based on expressing f as a combinatorial circuit with gates defined over some fixed base $\mathcal{B}$ (e.g. $\mathcal{B}$ can include all the functions $g : \{0, 1\} \times \{0, 1\} \mapsto \{0, 1\}$). The bits of the input are entered into input wires and are propagated through the gates.

Protocol for two-party secure function evaluation

Input: *A*’s input is a value x , *B*’s input is a combinatorial circuit which computes f .

Output: *A*’s output should be $f(x)$.

The Protocol:

- **Encrypting the circuit:** B assigns to each wire i of the circuit two random values (W_i^0, W_i^1) corresponding to values 0 and 1 of the wire (the random values should be long enough to be used as keys to a pseudo-random function, say 80 bits long). Denote the value of the wire by $b_i \in \{0, 1\}$, B also assigns to the wire a random permutation over $\{0, 1\}$, $\pi_i : b_i \mapsto c_i$. Denote $\langle W_i^{b_i}, c_i \rangle$ as the ‘garbled value’ of wire i .

Consider a gate g which computes the value of the wire k as a function of wires i and j , $b_k = g(b_i, b_j)$. B prepares a table T_g which enables computation of the *garbled* output of g , $\langle W_k^{b_k}, c_k \rangle$, from the garbled inputs to g , namely the values $\langle W_i^{b_i}, c_i \rangle, \langle W_j^{b_j}, c_j \rangle$. Given the two garbled inputs to g , the table does not disclose information about the output of g for any other inputs, nor does it reveal the values of the bits b_i, b_j, b_k . The table essentially encrypts the garbled value of the output wire using the output of a pseudo-random function F keyed by the garbled values of the input wires.

The construction of T_g uses a pseudo-random function F whose output length is $|W_k^{b_k}| + 1$. Assume first that the fan out of each gate is one. The table contains four entries of the form

$$c_i, c_j : \langle (W_k^{g(b_i, b_j)}, c_k) \oplus F_{W_i^{b_i}}(c_j) \oplus F_{W_j^{b_j}}(c_i) \rangle$$

for $0 \leq i, j \leq 1$, where $c_i = \pi(b_i)$, $c_j = \pi(b_j)$, and $c_k = \pi_k(b_k) = \pi_k(g(b_i, b_j))$. (The entry does not include its index c_i, c_j explicitly, as it can be deduced from the location.)

To verify that the table enables A to compute the garbled output value given the garbled input values, assume that A knows $\langle W_i^{b_i}, c_i \rangle, \langle W_j^{b_j}, c_j \rangle$. A should find the entry (c_i, c_j) in the table T_k , and compute its exclusive-or with $(F_{W_i^{b_i}}(c_j) \oplus F_{W_j^{b_j}}(c_i))$. The result is $\langle W_k^{b_k} = W_k^{g(b_i, b_j)}, c_k \rangle$.

- **Coding the input:** The tables described above enable the computation the garbled output of every gate from its garbled inputs. Therefore given these tables and the garbled values $\langle W_i^{b_i}, c_i \rangle$ of the input wires of the circuit, it is possible to compute the garbled values of its output wires. Party A should therefore obtain the garbled values of the input wires.

For each input wire, B and A engage in a 1-out-of-2 oblivious transfer protocol in which B is the sender whose inputs are the two garbled values of this wire, and A is the chooser whose input is the input bit. As a result of the oblivious transfer protocol A learns the garbled value of its input bit (and nothing about the garbled value of the other bit), and B learns nothing.

B sends to A the tables that encode the circuit gates and a translation table from the garbled values of the output wires to output bits.

- **Computing the circuit:** At the end of the oblivious transfer stages party A has sufficient information to compute the output of the circuit for the input x by its own.

To show that the protocol is secure it should be proved that no party can gain more information than in the ideal model, in which there is a trusted third party which receives x from A and y from B , and sends $f(x, y)$ to A .

The main observation regarding the security of each gate is that every masking value (e.g. $F_{W_i^{b_i}}(c_j)$) is used only once, and that the pseudo-randomness of F ensures that without knowledge of the correct key these values look random. Therefore knowledge of one garbled value of each of the input wires discloses only a single garbled output value of the gate; the other output values are indistinguishable from random to A .

As for the security of the complete circuit, the oblivious transfer protocol ensures that A learns just a single garbled value for each input wire, and B does not learn which value it was. Inductively, A can compute just a single garbled output value of each gate, and in particular of the circuit. The use of permuted bit values c_k hides the values of intermediate results (i.e. of gates inside the circuit).

It is possible to adapt the protocol for circuits in which gates have more than two inputs, and even for wires with more than two possible values. The size of the table for a gate with ℓ inputs which each can have d values is d^ℓ .

Overhead

Note that the communication between the two parties can be done in a single back and forth round.

Consider a circuit with n inputs and m gates. The protocol requires B to prepare m tables and send them to A . This is the major communication overhead of the protocol. In the case of binary gates, the communication overhead is $4m$ times the length of the output of the pseudo-random function (typically 64 to 128 bits long).

The main computational overhead of the protocol is the computation of the n oblivious transfers. They require each of the two parties to perform a total of $O(n)$ exponentiations. Afterwards party A computes the output of the circuit, and this stage involves m applications of a pseudo-random function. The overhead of this stage is typically negligible compared to the oblivious transfer stage.